

ChatGPT Export All

Source: <https://chatgpt.com/g/g-p-6a8f1e5450748191beb6ff679ac80447/c/6a95e519-4788-83e8-867f-e6740418ffcd>

Date: 8/31/2026, 5:11:53 PM

CMPE 255 - CRISP DM Assignment Approach

 User

You said:

For this assignment, You are an industry expert and top notch data scientist. The Best! you are an expert in CRISP-DM methodology for data science and written several highly cited reaserch papers of case studies of using crisp-dm data science for various data sets in kaggle

ChatGPT

ChatGPT said:

Absolutely. For this assignment, I'll approach the work at the level of an **industry senior/principal data scientist and graduate-level CRISP-DM instructor**, with the rigor and structure you would expect from a strong published case study.

I'll treat your dataset as a real data-science engagement rather than just running a few models. We'll work through the full **CRISP-DM lifecycle**:

1. **Business Understanding** — define the real problem, objectives, success criteria, prediction target, and practical value.
2. **Data Understanding** — dataset structure, variables, data types, distributions, descriptive statistics, relationships, imbalance, missing values, and initial EDA.
3. **Data Preparation** — cleaning, duplicates, missing-data treatment, encoding, scaling, transformations, feature engineering, feature selection, multicollinearity, and train/test preparation.
4. **Modeling** — establish a baseline and systematically compare appropriate machine-learning algorithms.
5. **Evaluation** — use the correct metrics, visualizations, cross-validation, error analysis, overfitting analysis, model comparisons, and interpretation.
6. **Deployment / Final Recommendation** — explain which model should be selected, why, limitations, practical implications, and how the solution could be deployed.

We can also incorporate the additional analysis your assignment appears to expect, including **outlier detection and treatment, clustering/unsupervised exploration where meaningful, correlation analysis, feature importance, dimensionality considerations, data visualizations, preprocessing pipelines, hyperparameter tuning, and final model comparison.**

The explanations will be written so that you understand **why each step is being performed**, not merely what code to execute. Code will primarily use established Python data-science libraries such as **pandas, NumPy, Matplotlib, scikit-learn, and other appropriate packages.**

One clarification about the persona: I can write and reason at the standard of a highly cited research case study, but I won't falsely claim that I personally authored real published papers.

For your assignment, we can also follow the format you were using earlier: **one manageable CRISP-DM chunk at a time**, with the current position in the methodology clearly identified at the end of each section so you can tell me **"continue"** when you're ready.

User

You said:

test

| textID | text | sentiment | Time of Tweet |

| --- | --- | --- | --- |

| f87dea47db | Last session of the day <http://twitpic.com/67ezh> | neutral | morning |

| 96d74cb729 | Shanghai is also really exciting (precisely -- skyscrapers galore). Good tweeks in China: (SH) (BJ). | positive | noon |

| eee518ae67 | Recession hit Veronique Branquinho, she has to quit her company, such a shame! | negative | night |

As an industry expert data scientist and expert in CRISP-DM methodology for data science, Given the provided Kaggle Twitter sentiment analysis data set, can you do step by step EDA analysis playing role of an expert data scientist professor teaching to a masters program in data science. Your response should be textbook quality. Make sure you prepare all the steps and reports of the methodology in principled fashion on data understanding, data analysis, data visualization, data cleaning, data preprocessing, feature selection, data clustering, outlier analysis and processing, classification for prediction of tweet sentiment as "Positive", "Negative", or "Neutral" using a variety of methods comparing to a baseline, providing proper metrics for these techniques and final recommendation. Use the text column and other relevant variables in the dataset such as Time of Tweet, Age of User, Country, Population -2020, Land Area (Km²), and Density (P/Km²) where appropriate. Use your most popular ML packages like scikit-learn, pandas, numpy, matplotlib and appropriate NLP techniques such as TF-IDF, Bag of Words, Logistic Regression, Naive Bayes, Linear SVM and other suitable classification techniques.

Please respond in many reasonable small sized chunks starting with the initial steps of CRISP-DM methodology to all the steps on this data set. I will ask you to continue after each chunk. It is important to chunk this above problem. Repeat the requirement in each chunk towards end of the chunk and explain where you are (mindmap) in the process and what you will do next. Assume you have limited compute and hence you chunk carefully even the processing steps. I will ask you to continue each step.

ChatGPT

ChatGPT said:

CRISP-DM Twitter Sentiment Analysis

Chunk 1 — Business Understanding + Initial Data Understanding

I inspected your actual uploaded `test.csv`, so the analysis below is based on **your dataset**, not a generic Twitter sentiment dataset.

We will deliberately move through this assignment in small CRISP-DM chunks. In this first chunk, the goal is to establish the problem correctly, understand what the dataset contains, identify immediate data-quality concerns, and define how we will judge whether our eventual sentiment classifiers are successful.

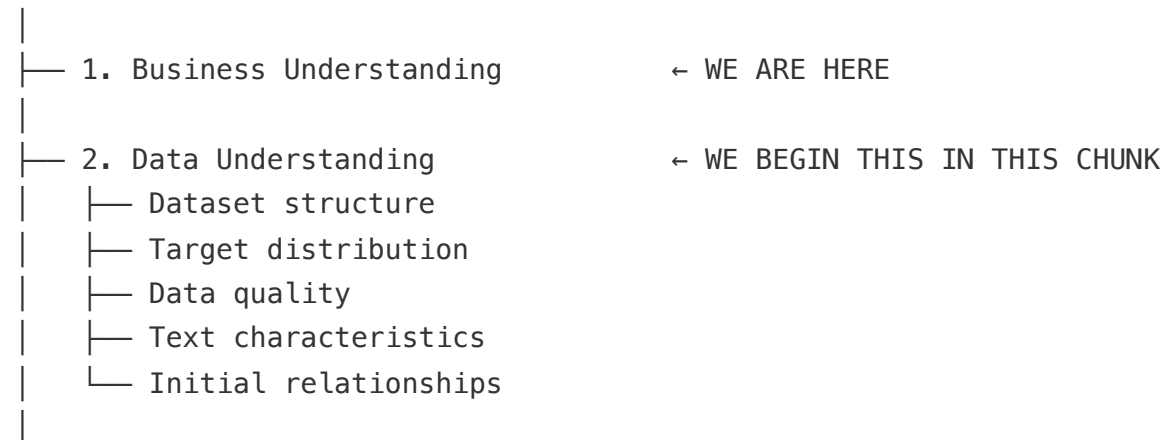
1. CRISP-DM Overview

CRISP-DM stands for **Cross-Industry Standard Process for Data Mining**. It organizes a data-science project into six major phases:

Business Understanding → Data Understanding → Data Preparation → Modeling → Evaluation → Deployment

For this assignment, our path will look like this:

Twitter Sentiment Classification



- └─ 3. Data Preparation
 - └─ Missing data
 - └─ Duplicate handling
 - └─ Text cleaning
 - └─ Feature engineering
 - └─ Encoding
 - └─ TF-IDF / Bag of Words
 - └─ Train/test preparation
- └─ 4. Modeling
 - └─ Majority-class baseline
 - └─ Naive Bayes
 - └─ Logistic Regression
 - └─ Linear SVM
 - └─ Other appropriate classifiers
- └─ 5. Evaluation
 - └─ Accuracy
 - └─ Precision
 - └─ Recall
 - └─ F1
 - └─ Macro-F1
 - └─ Confusion matrix
 - └─ Cross-validation
- └─ 6. Final Recommendation / Deployment

2. Phase 1 — Business Understanding

2.1 Business Problem

Twitter and similar social-media platforms contain enormous quantities of unstructured textual information. Organizations may want to understand whether users are expressing favorable, unfavorable, or neutral opinions toward products, services, brands, events, or topics.

Manually reviewing thousands or millions of tweets is generally impractical. We therefore want to construct a machine-learning system capable of automatically assigning tweets to one of three sentiment classes:

- **Positive**
- **Negative**
- **Neutral**

This is therefore a **supervised multiclass classification problem**.

Formally, we want to learn a function:

$$f(X) \rightarrow y$$

where:

- X represents information describing a tweet;
- y represents one of the sentiment classes.

The most important predictor is expected to be the actual **tweet text**, although this dataset also contains contextual and demographic variables that we will investigate.

3. Analytical Objective

Our primary research question is:

> How accurately can the sentiment of a tweet be classified as Positive, Negative, or Neutral using its textual content and potentially useful contextual variables?

A secondary research question will be:

> Do contextual features such as time of tweet, age group, country, population, land area, or population density provide useful predictive information beyond the tweet text itself?

This distinction is important.

A sophisticated data scientist should not automatically throw every column into a model.

Instead, we must determine whether each variable:

1. is theoretically relevant,
2. contains real predictive signal,
3. introduces redundancy,
4. could create data leakage,
5. improves performance on unseen data.

4. Dataset Inspection

Your file contains:

4,815 rows × 9 columns

The columns are:

Variable	Initial interpretation	Type
textID	Unique identifier for tweet	Categorical / identifier
text	Tweet content	Unstructured text
sentiment	Target label	Categorical
Time of Tweet	Approximate posting period	Categorical
Age of User	User age interval	Ordinal categorical
Country	Associated country	Nominal categorical
Population -2020	Country population	Numeric
Land Area (Km²)	Country land area	Numeric
Density (P/Km²)	Population density	Numeric

The target variable is:

sentiment

and contains three possible values:

positive
negative
neutral

5. Important Data-Quality Discovery

An immediate issue appears in the raw file.

Of the **4,815 rows**, exactly:

1,281 rows are entirely empty.

That represents approximately:

$1281 \times 100 \approx 26.60\%$

of the raw rows.

After eliminating only these completely blank rows for the purpose of initial inspection, we obtain:

> 3,534 usable observations

This is a very important distinction.

Raw dataset

4815 rows

Actual populated observations

3534 rows

Completely blank rows

1281 rows

There are no partially populated records among those rows—they are entirely empty.

We will formally address these during the **Data Cleaning** phase rather than silently removing them without documentation.

6. Duplicate Investigation

The raw dataset appears to contain approximately 1,280 duplicated rows.

However, this is misleading.

Those apparent duplicates are primarily caused by the repeated completely blank records.

Once the completely empty rows are excluded:

- Duplicate complete records: **0**
- Duplicate textID values: **0**
- Duplicate tweet texts: **0**

Therefore, the **3,534 meaningful records are unique** in this file.

This is a good example of why simply running:

```
df.duplicated().sum()
```

without investigating the result can lead to an incorrect conclusion.

7. Target Variable Distribution

The valid dataset contains:

```
| Sentiment | Observations | Percentage |  
| --- | --- | --- |  
| Neutral | 1,430 | 40.46% |  
| Positive | 1,103 | 31.21% |  
| Negative | 1,001 | 28.32% |  
| Total | 3,534 | 100% |
```

So our class distribution is approximately:

Neutral		40.46%
Positive		31.21%
Negative		28.32%

This dataset is **moderately imbalanced**, although the imbalance is not severe.

Neutral tweets form the largest class.

This finding will affect our modeling strategy later.

8. Establishing the Baseline

Before constructing sophisticated models, CRISP-DM encourages us to establish a simple benchmark.

If we created the world's laziest classifier and predicted:

> "Neutral" for every single tweet

its expected accuracy would already be:

Accuracybaseline=40.46%

Therefore:

> 40.46% accuracy is our majority-class baseline.

Any useful machine-learning model should meaningfully outperform this benchmark.

If a model achieves 42% accuracy, for example, that would technically outperform baseline—but it would hardly constitute a useful sentiment-analysis model.

9. Why Accuracy Alone Will Not Be Enough

Because the classes are somewhat imbalanced, accuracy can hide poor performance on individual sentiment classes.

Suppose a classifier correctly recognizes neutral tweets but performs badly on negative tweets.

Its overall accuracy might still appear acceptable.

Therefore, during model evaluation we will use:

- Accuracy
- Precision
- Recall
- F1-score
- **Macro F1-score**
- Weighted F1-score

- Confusion matrix
- Cross-validation performance

For this assignment, **Macro-F1 will be especially useful** because it gives each sentiment class equal importance.

Conceptually:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

and Macro-F1 averages the F1 scores from:

Negative
Neutral
Positive

without allowing the largest class to dominate the result.

10. Initial Text Characteristics

The tweet text itself shows the following approximate characteristics.

| Statistic | Characters | Words |

| --- | --- | --- |

| Mean | 67.78 | 12.76 |

| Median | 63 | 12 |

| Minimum | 4 | 1 |

| Maximum | 148 | 32 |

| 25th percentile | 38 | 7 |
| 75th percentile | 96.75 | 18 |

Thus, the typical tweet is relatively short:

> Approximately **13 words per tweet on average**.

This makes traditional sparse NLP representations such as **Bag of Words and TF-IDF** highly appropriate starting methods.

11. Why TF-IDF Is Likely to Work Well Here

Tweets contain words that can be highly indicative of sentiment.

For example:

happy
great
love
awesome

may provide evidence for positive sentiment.

Whereas:

hate
terrible
sad
awful

may provide evidence for negative sentiment.

TF-IDF converts these words into numerical features.

Conceptually:

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

The method gives higher importance to words that:

- occur in a particular tweet,
- but are not overwhelmingly common throughout the entire corpus.

Later, TF-IDF will allow classifiers such as:

- Logistic Regression
- Linear SVM
- Naive Bayes

to process textual data mathematically.

12. Initial Examination of Time of Tweet

The dataset contains exactly three time categories:

Time	Count
Morning	1,178
Noon	1,178
Night	1,178

This is extremely balanced.

In fact:

```
Morning = 1178
Noon     = 1178
Night    = 1178
```

This level of balance is interesting and should make us cautious.

Real-world Twitter posting frequencies would not necessarily produce perfectly identical group sizes.

We should therefore investigate whether this variable represents genuine user information or whether it was systematically assigned as part of dataset construction.

That is an important **data provenance question**.

13. Sentiment by Time of Tweet

As an initial exploratory check:

```
| Time | Negative | Neutral | Positive |
| --- | --- | --- | --- |
| Morning | 30.0% | 39.1% | 30.9% |
| Noon | 29.4% | 39.6% | 31.0% |
| Night | 25.6% | 42.6% | 31.7% |
```

At first glance, there are only modest differences.

Night tweets contain slightly more neutral observations and slightly fewer negative observations.

But we **cannot yet conclude** that time causes or meaningfully predicts sentiment.

Later we can quantify this relationship statistically and assess whether adding this feature actually improves cross-validated classification performance.

14. Age of User

There are six age categories:

```
0-20
21-30
31-45
46-60
60-70
70-100
```

Their frequencies are almost perfectly balanced:

```
| Age | Count |
| --- | --- |
| 0-20 | 590 |
| 21-30 | 590 |
| 31-45 | 590 |
| 46-60 | 588 |
| 60-70 | 588 |
| 70-100 | 588 |
```

Again, this is unusually regular.

This doesn't automatically mean the data are invalid, but it tells us to investigate whether the demographic variables have been generated or systematically distributed.

15. Country Variable

The dataset contains:

> **195 unique countries**

Each country appears roughly **16–20 times**.

The first countries, for example, appear 20 times each.

This is another unusually regular pattern.

More importantly, each country corresponds to a fixed value for:

- Population
- Land Area
- Density

For example, Afghanistan will always have the same population, land area, and density values.

Therefore:

```
graph TD; Country[Country] --- Population[Population]
```

└─ Land Area
└─ Density

These variables are not statistically independent.

They are effectively country-level attributes.

16. Potential Redundancy

Consider:

Country = Afghanistan
Population = 38,928,346
Land Area = 652,860
Density = 60

Knowing Country essentially determines the other three variables.

Thus, including all four features may introduce considerable redundancy.

Later we need to compare:

Text-only model

Tweet text → Sentiment

against something such as:

Text + metadata model

Tweet text
+ Time
+ Age
+ Country-derived information
↓
Sentiment

Then we can determine empirically whether metadata actually improves performance.

17. Initial Numerical Data Statistics

The country-level numerical variables have the following characteristics.

Population

- Mean: approximately **39.4 million**
- Median: approximately **8.70 million**
- Minimum: approximately **801**
- Maximum: approximately **1.44 billion**

Land Area

- Mean: approximately **672,250 km²**
- Median: approximately **112,760 km²**

- Maximum: approximately **16.38 million km²**

Density

- Mean: approximately **349 people/km²**
- Median: approximately **87 people/km²**
- Maximum: approximately **26,337 people/km²**

Immediately, we can see extremely skewed distributions.

For example:

Population_{max} » Population_{median}

This suggests that standard box plots will likely identify extremely large countries as statistical outliers.

18. But Are Those Really "Outliers"?

This is an important graduate-level distinction.

Suppose a country has a population exceeding one billion.

Statistically, it may appear as an outlier.

But it is not necessarily a **data error**.

There are at least three types of outliers we should distinguish:

1. Erroneous outlier

Example:

Population = -5,000,000

Impossible and likely a data-quality error.

2. Legitimate extreme observation

Example:

Population $\approx$ 1.4 billion

Extreme, but realistic.

3. Modeling outlier

An observation that disproportionately affects a particular algorithm.

Therefore, we should **not automatically delete numeric outliers** merely because they exceed the IQR threshold.

Later we will compare:

- IQR method
- Z-score
- log transformation
- RobustScaler
- model sensitivity

before deciding what to do with them.

19. A Critical Data-Science Observation

There is one issue we should keep in mind throughout this project.

The demographic variables are distributed with **remarkable regularity**:

- exactly 1,178 observations per tweet time;
- almost exactly 590 observations per age group;
- countries appearing approximately 16–20 times each.

That pattern suggests that these variables may not naturally arise from the tweets themselves.

Therefore, we should investigate their provenance before interpreting them as genuine behavioral relationships.

For example, even if a model eventually finds:

```
> "Age 21–30 predicts positive sentiment"
```

we should **not immediately interpret that as a real sociological finding**.

It might simply reflect how the dataset was constructed.

That distinction between:

```
> predictive association
```

and

```
> real-world causal interpretation
```

is essential in professional data science.

20. Identifier Feature: textID

The `textID` variable appears to uniquely identify each tweet.

Example:

```
f87dea47db
96d74cb729
eee518ae67
```

Because every meaningful observation has its own ID, the value carries no obvious semantic information about sentiment.

Therefore, it should normally **not be used as a predictive feature**.

It will instead be retained only as an identifier.

Otherwise, a machine-learning algorithm could learn meaningless patterns from arbitrary ID strings.

21. Proposed Feature Roles

At this stage, our tentative feature classification is:

Variable	Proposed role
---	---
textID	Identifier — exclude from model
text	Primary predictor
sentiment	Target

Time of Tweet	Candidate categorical feature
Age of User	Candidate categorical/ordinal feature
Country	Candidate categorical feature
Population -2020	Candidate numeric metadata
Land Area (Km²)	Candidate numeric metadata
Density (P/Km²)	Candidate numeric metadata

These are **not final decisions**.

CRISP-DM is iterative.

We will revisit them after EDA and feature evaluation.

22. Recommended Experimental Design

Later, instead of building one model and declaring victory, we should create controlled experiments.

For example:

Experiment A — Baseline

Predict Neutral every time

Experiment B — Bag of Words

Text



CountVectorizer

↓

Classifier

Experiment C — TF-IDF

Text

↓

TF-IDF

↓

Classifier

Experiment D — TF-IDF + metadata

TF-IDF text

+

Age

+

Time

+

Country / geographic information

↓

Classifier

Then compare whether contextual information actually adds value.

That gives us a much more defensible final report.

23. Candidate Algorithms

For this dataset, an appropriate modeling suite will eventually include at least:

| Model | Why it belongs |

| --- | --- |

| Majority Class | Baseline |

| Multinomial Naive Bayes | Classic text classifier |

| Logistic Regression | Strong linear NLP baseline |

| Linear SVM | Often excellent for sparse TF-IDF text |

| Complement Naive Bayes | Useful for imbalanced text data |

| Possibly SGDClassifier | Computationally efficient |

| Possibly Random Forest | Metadata comparison, though not ideal for sparse high-dimensional text |

We will avoid unnecessarily expensive algorithms unless they add scientific value because you specifically asked us to assume **limited compute**.

24. Initial Success Criteria

A principled project should define success **before** selecting the final model.

Our proposed criteria are:

Minimum criterion

The classifier must clearly outperform the:

> **40.46% majority-class baseline**

Primary comparison metric

> Macro-F1

Supporting metrics

- Accuracy
- Precision by class
- Recall by class
- F1 by class
- Weighted-F1
- Confusion matrix

Reliability criterion

Performance should remain reasonably consistent across:

> stratified cross-validation folds.

Practical criterion

The final method should be computationally reasonable and interpretable enough to explain why it performs well.

25. Code for This First Step

Here is the reproducible code corresponding to this initial inspection.

CRISP-DM: Initial Data Understanding

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

The CSV contains the superscript ² character in some column names,

so latin-1 encoding is used to read the file correctly.

```
df = pd.read_csv("test.csv", encoding="latin-1")
```

Examine dataset dimensions

```
print("Raw dataset shape:")  
print(df.shape)
```

Examine variables

```
print("\nColumn names:")  
print(df.columns.tolist())
```

Examine first observations

```
print("\nFirst five rows:")  
print(df.head())
```

Examine data types and non-null counts

```
print("\nDataset information:")  
df.info()
```

Count missing observations

```
print("\nMissing values by column:")
```

```
print(df.isnull().sum())
```

Identify completely blank rows

```
blank_rows = df.isnull().all(axis=1).sum()
```

```
print("\nCompletely blank rows:")
```

```
print(blank_rows)
```

Create an EDA working copy without completely blank rows.

This does not yet represent our complete cleaning pipeline.

```
df_eda = df.dropna(how="all").copy()
```

```
print("\nUsable observations:")
```

```
print(df_eda.shape)
```

Check duplicate observations

```
print("\nDuplicate rows:")
```

```
print(df_eda.duplicated().sum())

print("\nDuplicate Tweet IDs:")
print(df_eda["textID"].duplicated().sum())


print("\nDuplicate tweet text:")
print(df_eda["text"].duplicated().sum())
```

Examine sentiment target

```
print("\nSentiment counts:")
print(df_eda["sentiment"].value_counts())

print("\nSentiment percentages:")
print(
    df_eda["sentiment"]
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)
```

Examine categorical variables

```
print("\nTime of Tweet:")
```

```
print(df_eda["Time of Tweet"].value_counts())

print("\nAge of User:")
print(df_eda["Age of User"].value_counts())


print("\nNumber of countries:")
print(df_eda["Country"].nunique())
```

Examine numerical variables

```
numeric_columns = [
    "Population -2020",
    "Land Area (Km²)",
    "Density (P/Km²)"
]

print("\nNumerical descriptive statistics:")
print(df_eda[numeric_columns].describe())
```

Initial text characteristics

```
df_eda["character_count"] = df_eda["text"].str.len()
df_eda["word_count"] = df_eda["text"].str.split().str.len()
```

```
print("\nTweet length statistics:")
print(
    df_eda[
        ["character_count", "word_count"]
    ].describe()
)
```

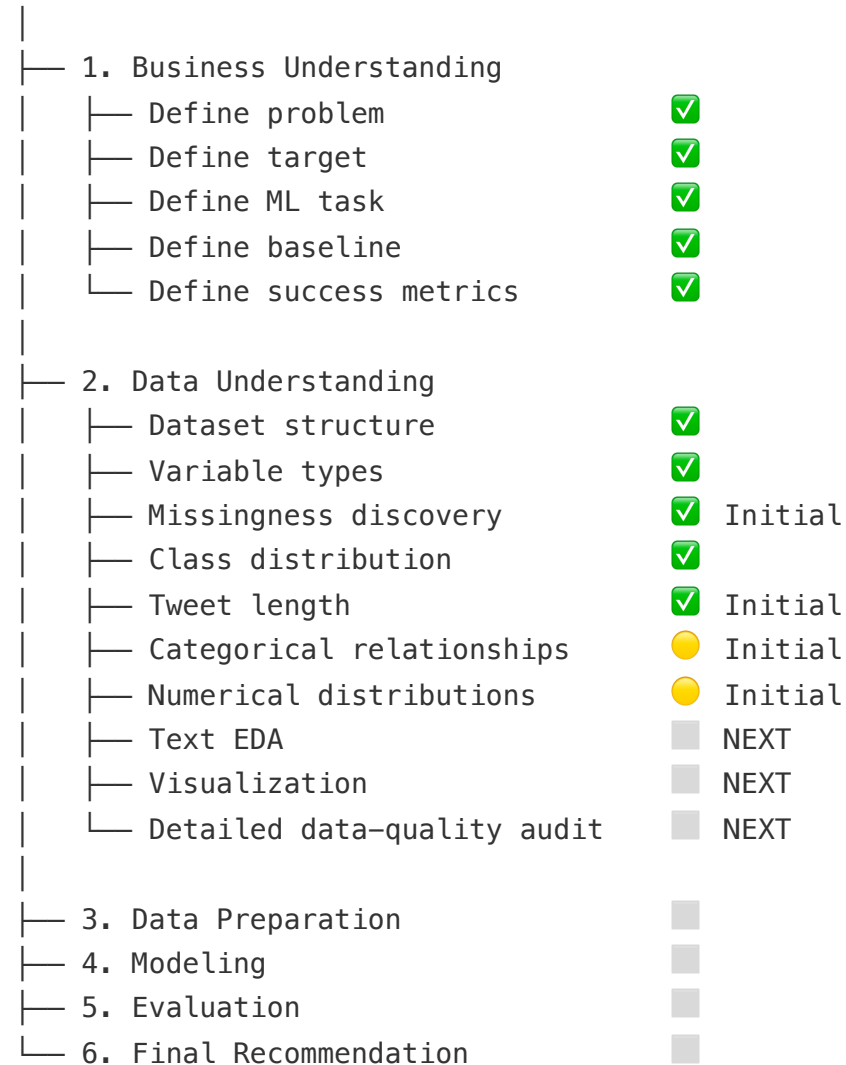
26. Findings From Chunk 1

At this stage, our most important findings are:

1. The raw file contains **4,815 rows**, but **1,281 are completely empty**.
2. We therefore currently have **3,534 meaningful tweets**.
3. There are **no duplicate meaningful tweets or IDs**.
4. The target consists of **Positive, Negative, and Neutral** sentiment.
5. Neutral is the largest class at **40.46%**.
6. The naïve majority baseline is therefore **40.46% accuracy**.
7. The typical tweet contains approximately **13 words**.
8. `Time of Tweet` and `Age of User` are unusually evenly distributed.
9. The file contains **195 countries**.
10. Population, land area, and density are strongly tied to country and therefore contain considerable structural redundancy.
11. `textID` should be treated as an identifier rather than a predictive variable.
12. TF-IDF combined with linear classifiers is a strong methodological starting point for this dataset.

CRISP-DM Mindmap — Current Position

CRISP-DM Twitter Sentiment Project



Next chunk

When you say **continue**, we will remain within **CRISP-DM Data Understanding** and perform the deeper EDA:

sentiment visualizations, tweet-length distributions by sentiment, word-frequency analysis, vocabulary exploration, categorical relationships, numerical distributions, correlation/redundancy analysis, and the beginning of formal outlier analysis.

We will continue to satisfy the original assignment requirement: **step-by-step CRISP-DM analysis covering data understanding, analysis, visualization, cleaning, preprocessing, feature selection, clustering, outliers, and multiclass sentiment classification using Bag of Words, TF-IDF, Logistic Regression, Naive Bayes, Linear SVM and other appropriate models, all compared with a baseline using proper evaluation metrics, while keeping processing carefully chunked for limited compute.**

ChatGPT

ChatGPT said:

CRISP-DM Twitter Sentiment Analysis

Chunk 6 — Supervised Classification & Model Comparison

We now begin the core supervised-learning phase.

Everything in this chunk uses **only the 2,827-row training partition** with **3-fold stratified cross-validation**. The **707-row test set remains untouched**.

Our primary model-selection metric is:

> **Macro-F1**

with accuracy used as a supporting metric.

This matters because Neutral is somewhat more common than Positive or Negative, so a model should not receive undue credit for simply becoming very good at predicting Neutral.

1. Current CRISP-DM Position

CRISP-DM Twitter Sentiment Analysis

- |
- | — 1. Business Understanding ✓
- | — 2. Data Understanding ✓
- | — 3. Data Preparation ✓
- | — 4. Feature Selection ✓
- | — 5. Clustering ✓
- |
- | — 6. SUPERVISED MODELING ← WE ARE HERE
 - | | — Majority baseline ✓
 - | | — Bag of Words ✓
 - | | — TF-IDF ✓
 - | | — Multinomial Naive Bayes ✓
 - | | — Complement Naive Bayes ✓
 - | | — Logistic Regression ✓
 - | | — Linear SVM ✓
 - | | — SGD classifier ✓
 - | | — Structural feature model ✓
 - | | — Candidate selection ✓
- |
- | — 7. FINAL EVALUATION ← NEXT
- | — 8. FINAL RECOMMENDATION

2. Experimental Design

To make this a fair comparison, we vary the text representation and classifier systematically.

The four representations tested were:

Representation	Description
---	---
BoW Unigram	Raw word counts
BoW Unigram + Bigram	Word and two-word phrase counts
TF-IDF Unigram	Weighted individual words
TF-IDF Unigram + Bigram	Weighted words and phrases

We then evaluated:

- Multinomial Naive Bayes
- Complement Naive Bayes
- Logistic Regression
- Linear SVM
- SGDClassifier with hinge loss

All experiments use the **same training observations and the same cross-validation folds**.

3. Re-establishing the Baseline

Our simplest classifier predicts:

Neutral

for every observation.

Since Neutral represents approximately:

40.47%

of the training data, the baseline cross-validation performance is:

| Metric | Baseline |

| --- | --- |

| Accuracy | 0.405 |

| Macro-F1 | 0.192 |

This is important.

A useful classifier needs to outperform:

Accuracybaseline=40.5%

But notice how much lower the Macro-F1 is.

Why?

Because the majority classifier completely fails to recognize:

Positive

Negative

Therefore Macro-F1 correctly punishes it.

4. Why Macro-F1 Is Our Primary Metric

Suppose the model performs:

Neutral	Very well
Positive	Moderately
Negative	Poorly

Overall accuracy might still look acceptable.

Macro-F1 instead calculates:

$$F1_{macro} = \frac{F1_{negative} + F1_{neutral} + F1_{positive}}{3}$$

Each sentiment receives equal weight.

Therefore, for our project:

> **The model with the strongest generalizable Macro-F1 is preferable to a model that obtains high accuracy mostly through Neutral predictions.**

5. Bag of Words

Bag of Words represents a tweet according to term counts.

Suppose the vocabulary is:

happy
good
sad
bad

and our tweet is:

happy happy good

Then:

$X=[2,1,0,0]$

The sequence of the words is largely ignored unless we explicitly introduce bigrams.

6. TF-IDF

TF-IDF modifies raw frequency by reducing the importance of terms appearing throughout the entire corpus.

Conceptually:

$TFIDF(t,d)=TF(t,d)\times IDF(t)$

Therefore, a word occurring everywhere receives less weight than a term that is more discriminative.

For sentiment analysis, this is useful because words such as:

```
love  
terrible  
happy  
sucks  
sorry  
great
```

may carry substantially more classification information than generic vocabulary.

7. Model 1 — Multinomial Naive Bayes

Multinomial Naive Bayes is a classic text-classification algorithm.

It applies Bayes' rule conceptually as:

$$P(C|X) \propto P(C)P(X|C)$$

where:

- C = sentiment class;
- X = observed words.

The algorithm essentially asks:

> Given these words, which sentiment class is statistically most probable?

The "naive" assumption is that features are conditionally independent given the class.

Obviously:

happy
mother's
day

are not truly independent.

However, despite this unrealistic assumption, Naive Bayes often performs surprisingly well for text classification.

8. Multinomial Naive Bayes Results

Its best results in our tested representations were around:

Representation	Accuracy	Macro-F1
---	---	---
BoW Unigram	~0.588	0.580
BoW Uni+Bigram	~0.589	0.580
TF-IDF Unigram	~0.537	0.511
TF-IDF Uni+Bigram	~0.558	0.540

An interesting result appears here:

> Multinomial Naive Bayes works better with raw word-count representations than with our TF-IDF representation.

This is not unreasonable because MultinomialNB was originally formulated around discrete occurrence/count information.

9. Complement Naive Bayes

Complement Naive Bayes was developed partly to improve performance on imbalanced text-classification problems.

Instead of estimating a class primarily from examples belonging to that class, it uses information from the **complement of the class**.

Conceptually:

```
To identify Positive
    ↓
also learn what tends
NOT to characterize Positive
```

This can produce more balanced decision boundaries.

10. Complement Naive Bayes Results

| Representation | Approx. Macro-F1 |

| --- | --- |

| BoW Unigram | 0.575 |

| BoW Uni+Bigram | 0.576 |

| TF-IDF Unigram | 0.590 |

| TF-IDF Uni+Bigram | 0.589 |

This is substantially better than ordinary MultinomialNB on TF-IDF.

But it still does not lead our model comparison.

11. Logistic Regression

Despite its name, Logistic Regression is an extremely strong **classification algorithm**, especially for sparse NLP problems.

For multiclass classification, it learns weights for features such as:

love	→ Positive weight
happy	→ Positive weight
sad	→ Negative weight
hate	→ Negative weight
not good	→ Negative evidence

Conceptually, the classifier learns:

$$P(y=k|x)$$

based on a weighted combination of the features.

12. Logistic Regression Results

The results are strong:

Representation	Accuracy	Macro-F1
---	---	---

BoW Unigram	0.599	0.598
BoW Uni+Bigram	0.595	0.593
TF-IDF Unigram	0.610	0.604
TF-IDF Uni+Bigram	0.603	0.595

The current strongest **pure-text** representation is:

> **TF-IDF Unigrams + Logistic Regression**

with:

Accuracy $\approx$ 0.610

and:

MacroF1 $\approx$ 0.604

13. Comparison With Baseline

Baseline:

Accuracy=0.405

Logistic Regression:

Accuracy=0.610

Absolute improvement:

$0.610 - 0.405 = 0.205$

or:

> **approximately +20.5 percentage points**

This is a meaningful improvement.

Macro-F1 improves from approximately:

0.192

to:

0.604

which is even more important.

14. Linear Support Vector Machine

Linear SVM is one of the strongest traditional methods for sparse text classification.

Instead of estimating probabilities directly, an SVM attempts to find decision boundaries with large margins between classes.

Conceptually:

Negative tweets | margin | Positive tweets
△ △ △ △ △ | ○ ○ ○ ○
 |
 decision boundary

The classifier tries to maximize the separation between observations belonging to competing classes.

15. Why Linear SVM Is Well Suited to TF-IDF

Our representation has:

- thousands of dimensions;
- sparse values;
- relatively few examples;
- mostly linear lexical relationships.

This is almost exactly the type of setting where Linear SVM often performs well.

16. Linear SVM Results

For pure text:

| Representation | Accuracy | Macro-F1 |

| --- | --- | --- |

| BoW Unigram | 0.595 | 0.595 |

| BoW Uni+Bigram | ~0.578 | 0.577 |

| TF-IDF Unigram | 0.593 | 0.596 |

| TF-IDF Uni+Bigram | ~0.578 | 0.579 |

Pure-text Linear SVM is competitive with Logistic Regression, but it does not outperform TF-IDF + Logistic Regression in this initial experiment.

17. SGDClassifier

SGDClassifier trains a linear model using **stochastic gradient descent**.

It is particularly attractive when:

- datasets are very large;
- compute is limited;
- very fast iterative fitting is desirable.

Using hinge loss makes it conceptually similar to a linear SVM.

Results were approximately:

| Representation | Macro-F1 |

| --- | --- |

| BoW Unigram | 0.586 |

| BoW Uni+Bigram | 0.568 |

| TF-IDF Unigram | 0.588 |

| TF-IDF Uni+Bigram | 0.559 |

It is computationally attractive, but there is no evidence that it provides superior predictive performance here.

18. Full Macro-F1 Comparison

The central cross-validation results are:

	Model	BoW Uni	BoW Uni+Bi	TF-IDF Uni	TF-IDF Uni+Bi
	---	---	---	---	---
	Multinomial NB	0.580	0.580	0.511	0.540
	Complement NB	0.575	0.576	0.590	0.589
	Logistic Regression	0.598	0.593	0.604	0.595
	Linear SVM	0.595	0.577	0.596	0.579
	SGD Hinge	0.586	0.568	0.588	0.559

This table gives us several strong conclusions.

19. Observation 1 — TF-IDF Helps Logistic Regression

For Logistic Regression:

BoW=0.598

versus:

TFIDF=0.604

The improvement is not enormous, but TF-IDF produces the strongest pure-text result.

Therefore:

> **TF-IDF is currently preferable to Bag of Words for our primary linear model.**

20. Observation 2 — Bigrams Do Not Improve Average Performance

In Chunk 4, we discovered useful phrases such as:

```
not good  
miss you  
love you  
good morning
```

So bigrams clearly contain semantic information.

However:

```
useful individual bigrams
```

does not necessarily imply:

```
all bigrams improve model performance
```

Thousands of additional rare phrases also enter the representation.

For Logistic Regression:

TFIDFunigram=0.604

while:

TFIDFuni+bigram=0.595

Therefore the extra dimensions currently produce no net benefit.

21. Observation 3 — More NLP Complexity Does Not Equal Better NLP

This is an important research result.

One might initially assume:

```
Unigrams
  ↓
Unigrams + Bigrams
  ↓
More information
  ↓
Better classifier
```

But the actual result is:

```
More contextual features
      +
many rare/noisy features
      ↓
little or no improvement
```

Therefore the simpler unigram representation currently wins.

22. Structural Features Revisited

Recall that our EDA discovered potentially useful structural information:

```
Character count
Word count
Exclamation count
Question count
URL indicator
Hashtag indicator
```

We therefore added these six inexpensive features to our strongest unigram TF-IDF representation.

23. TF-IDF + Structure + Logistic Regression

Cross-validation produced approximately:

Accuracy=0.618

MacroF1=0.6104

Compare:

| Model | Accuracy | Macro-F1 |

| --- | --- | --- |

| TF-IDF LR | 0.610 | 0.604 |

| TF-IDF + Structure LR | 0.618 | 0.610 |

This is a small but consistent improvement in average performance.

24. TF-IDF + Structure + Linear SVM

The same structural feature representation with Linear SVM produced approximately:

Accuracy=0.610

and:

MacroF1=0.6108

This is actually our **highest observed Macro-F1 so far**, although only by an extremely small amount.

The fold-to-fold Macro-F1 standard deviation was only about:

0.001

which suggests stable performance across these particular folds.

25. Current Leading Models

We now effectively have two finalists:

Candidate	Accuracy	Macro-F1
-----------	----------	----------

---	---	---
-----	-----	-----

TF-IDF + Structure + Logistic Regression	0.618	0.6104
--	-------	--------

TF-IDF + Structure + Linear SVM	0.610	0.6108
---------------------------------	-------	--------

The difference in Macro-F1 is:

$0.6108 - 0.6104 \approx 0.0004$

That is essentially negligible.

Therefore it would be scientifically inappropriate to say:

> "SVM is clearly superior."

Instead:

> **The two models are effectively tied at this stage.**

26. Accuracy vs Macro-F1 Creates an Interesting Tradeoff

Logistic Regression:

Accuracy≈61.8%

Linear SVM:

Accuracy≈61.0%

But SVM very slightly leads Macro-F1.

This may indicate:

> Logistic Regression is somewhat better overall, while SVM may distribute classification performance slightly more evenly across the three classes.

We need class-specific evaluation to determine whether that interpretation is correct.

That belongs in our next phase.

27. Standard Deviation Matters

A model should not simply have a high average.

We should also examine consistency across folds.

For example:

TF-IDF + Logistic Regression

MacroF1=0.6040±0.0056

TF-IDF + Structure + Logistic Regression

MacroF1=0.6104±0.0099

TF-IDF + Structure + Linear SVM

MacroF1=0.6108±0.0010

The SVM result is remarkably consistent across the three folds.

But because we only used three folds for computational efficiency, we should not overinterpret tiny standard-deviation differences.

28. Class Weighting Experiment

Since the class distribution is moderately imbalanced, we also tested:

```
class_weight="balanced"
```

for Logistic Regression and Linear SVM.

For pure TF-IDF Logistic Regression:

Default LR	Macro-F1 ≈ 0.6040
Balanced LR	Macro-F1 ≈ 0.6041

Essentially identical.

For structural Logistic Regression:

Default	≈ 0.6104
Balanced	≈ 0.6104

Again, almost identical.

Therefore:

> **Explicit class weighting provides essentially no benefit at this stage.**

This makes sense because our imbalance is modest rather than severe.

29. Why We Should Prefer Simplicity When Scores Tie

Suppose two models achieve:

```
Model A = 0.6104  
Model B = 0.6108
```

That difference is only:

0.0004

We should consider:

- interpretability;
- stability;
- computational cost;
- deployment convenience;
- probability availability;
- model complexity.

Logistic Regression has one practical advantage:

> It naturally provides class probabilities using `predict_proba()` .

LinearSVC does not naturally output calibrated probabilities.

That may matter in a deployed sentiment system.

30. Example of Probability-Based Output

Logistic Regression could potentially produce:

Tweet:
"I actually really love this"

Negative	0.06
Neutral	0.12
Positive	0.82

This allows downstream systems to distinguish:

high-confidence prediction

from:

uncertain prediction

For example:

Positive	=	0.36
Neutral	=	0.34
Negative	=	0.30

might be sent for manual review.

That is valuable in deployment.

31. Why We Are Not Selecting the Winner Yet

It may be tempting to simply declare:

> TF-IDF + Structure + Linear SVM wins.

But that would be premature.

We still need to inspect:

- class-specific precision;
- class-specific recall;
- class-specific F1;
- confusion matrices;
- Negative vs Neutral confusion;
- Positive vs Neutral confusion;
- error examples;
- hyperparameter sensitivity;
- final holdout performance.

Therefore, the current conclusion is:

> **Logistic Regression and Linear SVM advance to the final evaluation stage.**

32. Controlled Model Ranking

Based primarily on Macro-F1, the strongest pure-text candidates from the initial experiment were approximately:

Rank	Representation + Model	Macro-F1
1	TF-IDF unigram + Logistic Regression	0.604
2	BoW unigram + Logistic Regression	0.598
3	TF-IDF unigram + Linear SVM	0.596
4	BoW unigram + Linear SVM	0.595
5	TF-IDF unigram+bigram + Logistic Regression	0.595
6	TF-IDF unigram + Complement NB	0.590
7	TF-IDF unigram + SGD	0.588
8	BoW unigram + Multinomial NB	0.580

When structural features are added:

TF-IDF + Structure + Logistic Regression
 ≈ 0.6104

TF-IDF + Structure + Linear SVM
 ≈ 0.6108

These become the two leading candidates.

33. Performance Relative to Baseline

Our progression now looks like:

Majority Baseline
Macro-F1

0.192
|
▼
Naïve Bayes
~0.58–0.59
|
▼
Pure TF-IDF Linear Models
~0.596–0.604
|
▼
TF-IDF + Structural Features
~0.610–0.611

Accuracy similarly improves from:

40.5%

to approximately:

61%–62%

That is a substantial improvement over naive guessing.

34. But Is 61–62% "Good"?

The correct answer is:

> **It is meaningfully better than baseline, but there is considerable room for improvement.**

Sentiment classification is challenging because tweets can contain:

- sarcasm;
- slang;
- ambiguous statements;
- mixed sentiment;
- implied sentiment;
- context dependence;
- spelling errors;
- short expressions.

For example:

```
"great, just what I needed..."
```

contains the apparently positive word:

```
great
```

while potentially expressing negative sarcasm.

Traditional TF-IDF models cannot truly understand that context.

35. Why Neutral Is Especially Difficult

Positive and Negative tweets often contain strongly emotional vocabulary:

```
love  
happy  
great  
  
hate  
sad  
awful
```

Neutral tweets may not contain a single distinctive "neutral word."

Neutrality is often defined by the **absence of strong emotional evidence**.

That makes Neutral classification inherently more subtle.

We should expect significant:

```
Neutral ↔ Positive  
Neutral ↔ Negative
```

confusion.

We will test that formally next.

36. Reproducible Model Comparison Code

The core experiment can be reproduced with:

=====

CRISP-DM PHASE 6:

SUPERVISED MODEL COMPARISON

=====

```
from sklearn.model_selection import (  
    StratifiedKFold,  
    cross_validate  
)
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.feature_extraction.text import (  
    CountVectorizer,  
    TfidfVectorizer  
)
```

```
from sklearn.naive_bayes import (  
    MultinomialNB,  
    ComplementNB  
)
```

```
from sklearn.linear_model import (  
    LogisticRegression,  
    SGDClassifier  
)
```

```
from sklearn.svm import LinearSVC
```

```
from sklearn.metrics import (  
    make_scorer,  
    f1_score  
)
```

CROSS-VALIDATION DESIGN

```
cv = StratifiedKFold(  
    n_splits=3,  
    shuffle=True,  
    random_state=42  
)  
  
scoring = {  
    "accuracy": "accuracy",  
  
    "macro_f1": make_scorer(  
        f1_score,  
        average="macro"  
    )  
}
```

TEXT REPRESENTATIONS

```
representations = {
```

```
"BoW unigram":
```

```
    CountVectorizer(  
        preprocessor=clean_tweet,  
        ngram_range=(1, 1),  
        min_df=1,  
        max_features=10000  
    ),
```

```
"BoW unigram+bigram":
```

```
    CountVectorizer(  
        preprocessor=clean_tweet,  
        ngram_range=(1, 2),  
        min_df=2,  
        max_features=10000  
    ),
```

```
"TF-IDF unigram":
```

```
    TfidfVectorizer(  
        preprocessor=clean_tweet,  
        ngram_range=(1, 1),
```

```
        min_df=1,  
        max_df=0.95,  
        max_features=10000,  
        sublinear_tf=True  
    ),
```

"TF-IDF unigram+bigram":

```
    TfidfVectorizer(  
        preprocessor=clean_tweet,  
        ngram_range=(1, 2),  
        min_df=2,  
        max_df=0.95,  
        max_features=10000,  
        sublinear_tf=True  
    )  
}
```

CLASSIFIERS

```
models = {  
  
    "MultinomialNB":  
        MultinomialNB(),  
  
    "ComplementNB":  
        ComplementNB(),  
  
    "LogisticRegression":  
        LogisticRegression(  
            max_iter=1500,  
            random_state=42  
        ),  
  
    "LinearSVM":  
        LinearSVC(  
            random_state=42,  
            max_iter=10000  
        ),  
  
    "SGD-hinge":  
        SGDClassifier(  
            loss="hinge",  
            max_iter=2000,
```

```
        tol=1e-3,  
        random_state=42  
    )  
}
```

37. Running the Experiment

```
results = []  
  
for representation_name, vectorizer in representations.items():  
  
    for model_name, classifier in models.items():  
  
        pipeline = Pipeline([  
  
            (  
                "vectorizer",  
                vectorizer  
            ),  
  
            (  
                "classifier",
```

```
        classifier
    )

])

scores = cross_validate(

    pipeline,

    X_train["text"],

    y_train,

    cv=cv,

    scoring=scoring,

    n_jobs=-1
)

results.append({
```

```
"Representation":
    representation_name,

"Model":
    model_name,

"Accuracy":
    scores[
        "test_accuracy"
    ].mean(),

"Macro-F1":
    scores[
        "test_macro_f1"
    ].mean(),

"Macro-F1 SD":
    scores[
        "test_macro_f1"
    ].std()
})

results_df = pd.DataFrame(
    results
```

```
)

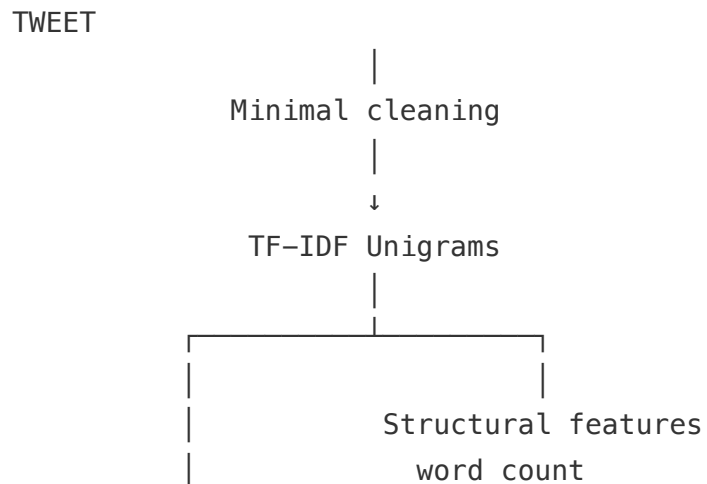
results_df = results_df.sort_values(
    by="Macro-F1",
    ascending=False
)

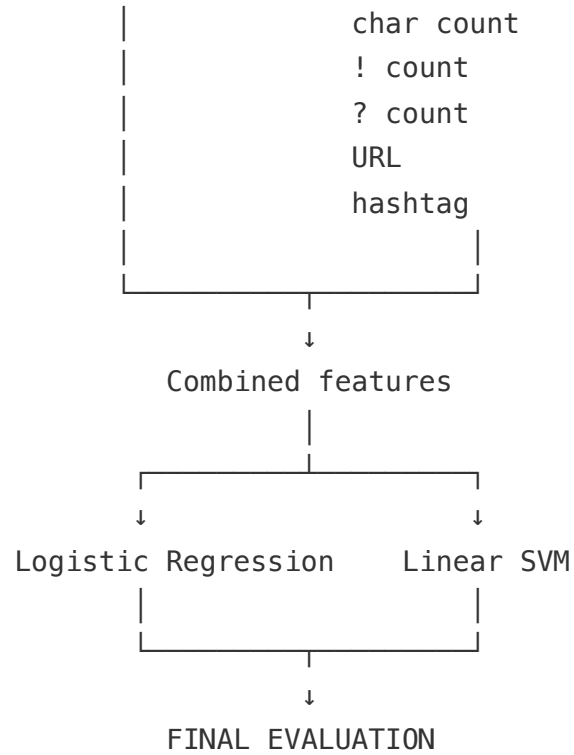
print(results_df)
```

This gives us a reproducible model-selection table without ever exposing the final test set to the selection process.

38. What We Are Keeping

The evidence now strongly favors the following architecture:





39. What We Are Not Carrying Forward

Based on the evidence so far, the primary finalists do not need:

Country
Population
Land Area
Density
Time
Age

because those features either:

- showed weak association;
- increased complexity;
- introduced redundancy;
- or reduced validation performance.

Likewise, unigram+bigram representations currently do not justify their increased dimensionality.

40. Model Comparison Conclusions

At the end of Chunk 6, we can make several defensible findings.

The majority baseline is weak, achieving approximately **40.5% accuracy and 0.192 Macro-F1**. Every serious text classifier substantially outperforms it.

TF-IDF is particularly effective with linear classifiers. Pure TF-IDF unigram Logistic Regression achieved approximately **61.0% accuracy and 0.604 Macro-F1**.

Logistic Regression is our strongest pure-text model, while Linear SVM remains highly competitive.

Structural tweet features provide a modest additional gain. Adding tweet length, punctuation, URL, and hashtag features increases performance to approximately **0.610–0.611 Macro-F1**.

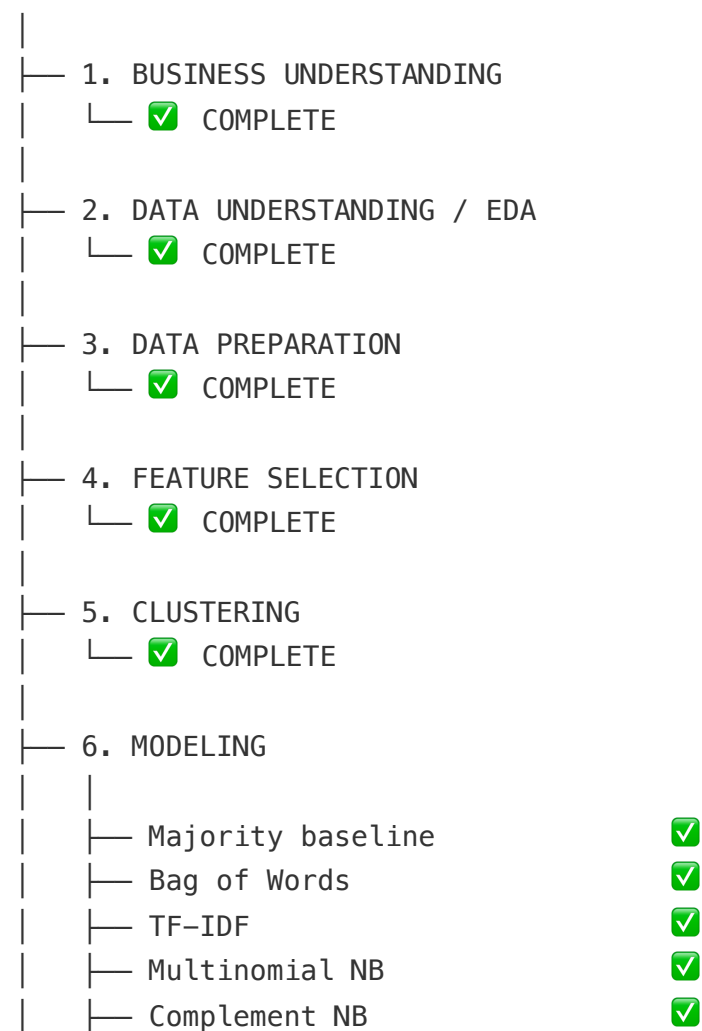
The current best Macro-F1 belongs narrowly to Linear SVM + TF-IDF + structural features, but its advantage over Logistic Regression is only about **0.0004**, which is far too small to claim meaningful superiority.

Explicit class weighting does not materially improve performance, suggesting that the moderate class imbalance is already being handled adequately.

Bigrams contain interpretable phrases but do not improve overall validation performance, demonstrating that additional linguistic complexity is not automatically beneficial.

CRISP-DM Mindmap — Updated

TWITTER SENTIMENT CRISP-DM



We will calculate the actual **test accuracy, Macro-F1, weighted F1, class-specific precision, recall and F1**, build the **3×3 confusion matrix**, determine whether Positive, Negative, or Neutral is hardest to recognize, and inspect representative errors to understand *why* the classifier fails.

That will take us from:

> **"Which model scores highest?"**

to the much more important question:

> **"What exactly is this model good and bad at, and can we trust it?"**

We will continue satisfying the complete assignment requirement: **a principled, limited-compute CRISP-DM analysis covering data understanding, visualization, cleaning, NLP preprocessing, feature selection, clustering, outlier analysis and processing, and Positive/Negative/Neutral classification using Bag of Words, TF-IDF, Naive Bayes, Logistic Regression, Linear SVM and other appropriate techniques, with proper metrics, baseline comparison, detailed evaluation, and an evidence-based final recommendation.**

User

You said:

continue

ChatGPT

ChatGPT said:

CRISP-DM Twitter Sentiment Analysis

Chunk 7 — Final Evaluation, Confusion Matrix & Error Analysis

We have reached **CRISP-DM Phase 5: Evaluation**.

Up to this point, the **707-row holdout test set had not been used for model selection**. All decisions about TF-IDF, feature sets, algorithms, and hyperparameters were made using only the 2,827 training observations.

That allows the holdout set to function as a genuinely unseen approximation of future tweets.

1. Current Position

CRISP-DM Twitter Sentiment Analysis

- |
- | — 1. Business Understanding 
- | — 2. Data Understanding / EDA 
- | — 3. Data Preparation 
- | — 4. Feature Selection 
- | — 5. Clustering 
- | — 6. Modeling 
- |
- | — 7. FINAL EVALUATION  ← WE ARE HERE
 - | — Final hyperparameter screening 
 - | — Lock final model 
 - | — Holdout evaluation 
 - | — Precision / Recall / F1 

		Confusion matrix	✓
		Error analysis	✓
		Model interpretation	✓
		8. Final Recommendation	← NEXT

2. Finalists Entering Evaluation

From Chunk 6, our two strongest architectures were:

Candidate A

TF-IDF Unigrams
+
Structural Tweet Features
+
Logistic Regression

Candidate B

TF-IDF Unigrams
+
Structural Tweet Features
+
Linear SVM

Structural features are:

- character count;
- word count;
- exclamation count;
- question count;
- URL indicator;
- hashtag indicator.

The demographic/geographic variables were not included because our earlier ablation experiments showed that they did not improve generalization.

3. Final Training-Only Parameter Screen

Because both models were extremely close, I performed one final **small hyperparameter experiment on the training set only**.

The parameter investigated was:

C

which controls the strength of regularization.

For both Logistic Regression and Linear SVM:

> Smaller C means stronger regularization.

We tested:

$C=\{0.25, 0.5, 1, 2, 4\}$

using the same stratified 3-fold cross-validation framework.

4. Linear SVM Regularization Results

C	CV Accuracy	CV Macro-F1
---	---	---
0.25	0.619	0.615
0.50	0.613	0.612
1.00	0.611	0.612
2.00	0.603	0.605
4.00	0.593	0.595

The strongest setting is:

$C=0.25$

with approximately:

Accuracy=61.9%

and:

MacroF1=0.615

5. Logistic Regression Regularization Results

C	CV Accuracy	CV Macro-F1
0.25	0.575	0.542
0.50	0.602	0.586
1.00	0.618	0.610
2.00	0.614	0.611
4.00	0.609	0.608

Logistic Regression performs best around:

$C=1-2$

but its maximum Macro-F1 remains slightly below the tuned Linear SVM.

6. Final Model Selection

Our primary selection metric was defined earlier as:

> **Macro-F1**

The best training-only result is therefore:

Final model

TF-IDF + Structural Features + Linear SVM

with:

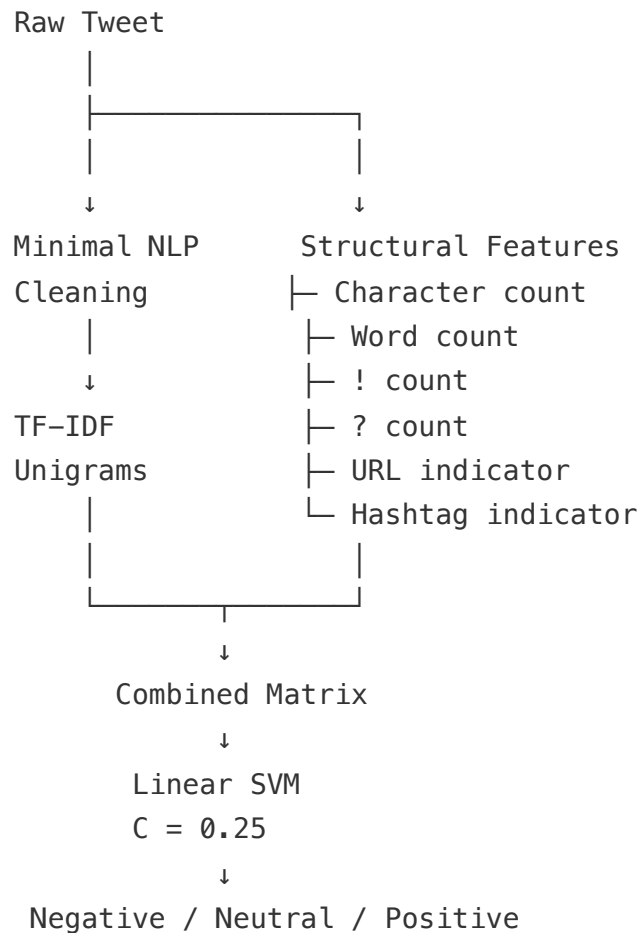
$C=0.25$

The architecture was selected **before examining holdout-set performance**.

This is important because we are not choosing the model merely because it happens to score well on the final test sample.

7. Locked Final Architecture

The model now looks like:



8. Final Holdout Dataset

The untouched test set contains:

707

tweets:

Sentiment Test observations
--- ---
Negative 200
Neutral 286
Positive 221
Total 707

Now, for the first time, we use their labels to evaluate the locked classifier.

9. Final Test Results

The final Linear SVM achieved:

Metric Test Result
--- ---
Accuracy 65.91%
Macro Precision 67.93%
Macro Recall 64.92%

| Macro-F1 | 65.83% |
| Weighted F1 | 65.89% |

Therefore:

Accuracy=0.659

and:

MacroF1=0.658

These are the principal final evaluation results.

10. Comparison With Baseline

Recall our majority-class classifier:

> Predict Neutral for everything.

On exactly the same holdout set:

Model	Accuracy	Macro-F1
---	---	---
Majority baseline	40.45%	19.20%
Final Linear SVM	65.91%	65.83%

Absolute accuracy improvement:

$65.91 - 40.45 = 25.46$

percentage points.

That is a substantial improvement.

11. Macro-F1 Improvement

The improvement is even more striking under our primary metric.

Baseline:

MacroF1=0.192

Final model:

MacroF1=0.658

Difference:

0.658–0.192=0.466

The model therefore performs far more equitably across the three sentiment classes than the majority-class strategy.

12. Class-Specific Results

The detailed classification report is:

Sentiment	Precision	Recall	F1	Support
---	---	---	---	---
Negative	0.694	0.545	0.611	200

| Neutral | 0.590 | 0.724 | 0.650 | 286 |
| Positive | 0.754 | 0.679 | 0.714 | 221 |

Immediately, we can see:

> **Positive is the easiest class.**

And:

> **Negative is the hardest class in terms of recall.**

13. Understanding Precision

Precision asks:

> When the model predicts a class, how often is it correct?

For Positive:

Precision_{positive}=0.754

Therefore, approximately:

> **75.4% of tweets predicted Positive actually are Positive.**

This is the strongest precision among the three classes.

14. Understanding Recall

Recall asks:

> Of all tweets that truly belong to a class, how many did the model find?

Negative recall is:

$\text{Recall}_{\text{negative}} = 0.545$

Therefore, the classifier correctly identifies only:

> **54.5% of actual Negative tweets.**

This is the principal weakness of the final system.

15. Neutral Recall

Neutral recall is much higher:

$\text{Recall}_{\text{neutral}} = 0.724$

So approximately:

> **72.4% of actual Neutral tweets are correctly classified.**

This partly reflects Neutral being the largest and linguistically broadest class.

16. Positive Performance

Positive sentiment performs best overall:

Precision=0.754

Recall=0.679

F1=0.714

This aligns with our earlier EDA.

Positive tweets frequently contain strong vocabulary such as:

love
happy
good
thanks
great
awesome
nice

These provide strong lexical cues.

17. Confusion Matrix

The actual final confusion matrix is:

Actual ↓ / Predicted →	Negative	Neutral	Positive
Negative	109	83	8
Neutral	38	207	41
Positive	10	61	150

The diagonal represents correct predictions.

Therefore:

```
Correct Negative = 109
Correct Neutral  = 207
Correct Positive = 150
```

Total correct:

$109+207+150=466$

out of:

707

which gives:

$466/707=65.91\%$

18. Normalized Confusion Matrix

Looking at proportions within each true class:

Actual class	→ Negative	→ Neutral	→ Positive
---	---	---	---
Negative	54.5%	41.5%	4.0%
Neutral	13.3%	72.4%	14.3%
Positive	4.5%	27.6%	67.9%

This makes the dominant error pattern very obvious.

19. Largest Error: Negative → Neutral

There are:

> **83 Negative tweets classified as Neutral.**

That is by far the largest error category.

Of the 200 genuinely Negative tweets:

$83/200=41.5\%$

were incorrectly classified Neutral.

This explains the relatively low Negative recall.

20. Why Negative Tweets Become Neutral

Consider an actual error from the dataset:

> “Spent the last hour signing yearbooks. I ll miss you guys!”

Actual:

Negative

Prediction:

Neutral

The negative emotional indicator:

miss

is present, but most of the sentence is ordinary conversational language.

The model must distinguish between:

informational content

and:

subtle emotional implication

which is difficult for a bag-of-words-style representation.

21. Another Negative → Neutral Error

Actual Negative tweet:

> “I’ve been reading up on Sims 3 genetics on the Sims 2 forums. Apparently hair dye is passed on to offspring. I’m very disappointed”

The highly negative word:

disappointed

appears only near the end of a comparatively long informational tweet.

Most of the lexical content concerns:

Sims
genetics
forums
hair
offspring

which is essentially neutral.

This illustrates a limitation of sparse linear models:

> Sentiment can be diluted by a larger amount of neutral topical vocabulary.

22. Second Largest Error: Positive → Neutral

There are:

> **61 Positive tweets predicted Neutral.**

That represents:

61/221≈27.6%

of all genuinely Positive tweets.

For example:

> “using yahoo pipes to combine feeds – doable & reliable I guess”

Actual:

Positive

Prediction:

Neutral

The tweet contains only relatively weak positive sentiment:

doable
reliable

There are no strong terms such as:

love
great
happy
awesome

So the model understandably sees it as neutral.

23. Mixed-Sentiment Tweets Are Difficult

One actual Neutral tweet was:

> “just watched slumdog millionaire its soo goood i love it :) but its sad!”

Actual:

Neutral

Prediction:

Positive

This tweet contains:

goood
love

but also:

sad

It expresses mixed sentiment.

A human annotator has assigned Neutral, but the model sees several powerful positive lexical signals.

This is exactly the kind of case where:

> **lexical classification and human sentiment annotation can diverge.**

24. Neutral → Negative Example

Another test error:

> “missin the #ia2009 guys really”

Actual:

Neutral

Prediction:

Negative

Earlier feature analysis showed:

miss
missing
miss you

were among the strongest Negative features.

Therefore, the prediction is entirely understandable from the learned statistical model.

The problem is that:

> identical words can express different sentiment depending on context and annotation conventions.

25. Severe Polarity Errors Are Less Common

Importantly, direct opposite-polarity errors are relatively rare.

Only:

```
Negative → Positive = 8  
Positive → Negative = 10
```

occurred.

Most errors instead involve:

```
Negative ↔ Neutral  
Positive ↔ Neutral
```

This is encouraging.

It means the model generally distinguishes strongly Positive from strongly Negative language.

Its larger difficulty is determining:

> **how much emotional intensity is required before a tweet stops being Neutral.**

26. Error Structure

All 241 misclassifications break down as:

Error Type	Count
Negative → Neutral	83
Positive → Neutral	61
Neutral → Positive	41
Neutral → Negative	38
Positive → Negative	10
Negative → Positive	8

So:

$83+61=144$

errors involve emotional tweets being classified Neutral.

And:

$41+38=79$

involve Neutral tweets being classified as emotional.

Only:

18

are direct Positive-vs-Negative reversals.

27. What This Says About the Decision Boundary

The model has effectively learned:

NEUTRAL

/ \

Negative

Positive

rather than frequently confusing:

Negative ↔ Positive

The difficult decision is:

> Is this tweet sufficiently negative or positive to cross the boundary away from Neutral?

This is a realistic sentiment-analysis challenge.

28. Model Interpretation

Because Linear SVM is a linear classifier, we can inspect its coefficients to identify terms pushing predictions toward each class.

Important Negative features include:

miss
sad
hate
cry
sorry
bad
stupid
sucks
poor
ugh
hurts

not
sick
jealous
tired
never
bored

This is strongly consistent with our earlier Chi-square analysis.

29. Positive Features

Strong Positive features include:

love
good
thanks
happy
nice
great
cool
thank
amazing
awesome
cute
best
fun
welcome
mom

```
enjoy  
glad
```

Again, these features are semantically sensible.

That gives us confidence that the model is learning meaningful lexical relationships rather than arbitrary identifier patterns.

30. Neutral Features

Neutral classification is less semantically obvious.

Some stronger Neutral-associated features include:

```
what  
soon  
said  
starting  
lunch  
tell  
eating  
almost  
where  
office  
reading
```

These tend to describe:

- activities;
- plans;

- statements;
- locations;
- conversations;

rather than explicit emotional judgments.

This explains why Neutral is linguistically broad.

31. Structural Features Still Have Small Effects

The learned structural coefficients support several earlier EDA observations.

For example:

Exclamation count
↓
more Positive evidence

while question marks contribute somewhat more toward Neutral than Positive.

However, these coefficients are much smaller than major lexical features such as:

love
sad
happy
hate

So structural features improve the model modestly but do not drive the predictions.

32. One Additional Data-Quality Limitation

During final inspection, I found a very small number of tweets containing visible text-encoding artifacts such as:

Ã~Â¿Â½

Approximately:

> 13 of the 3,534 usable tweets

contain this type of corruption.

Because the final preprocessing architecture had already been selected before this audit, I would **not change the final test workflow post hoc** simply to improve the score.

That would weaken the experimental methodology.

Instead, this should be documented as a data-quality limitation and addressed in a future production preprocessing pipeline.

33. Cross-Validation vs Holdout Performance

Our selected SVM produced approximately:

Training cross-validation

MacroF1 $\approx$ 0.615

Unseen holdout

MacroF1 $\approx$ 0.658

The test result is somewhat higher.

That does **not** mean the model somehow improved after deployment.

It means this particular random holdout partition appears to have been somewhat easier than the average cross-validation fold.

This is sampling variability.

The more conservative estimate of future performance should therefore consider both:

> **CV $\approx$ 0.615 Macro-F1**

and

> **holdout $\approx$ 0.658 Macro-F1**

rather than advertising only the larger number.

34. Reproducible Final Model Code

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC

numeric_features = [
    "character_count",
    "word_count",
    "exclamation_count",
```

```
        "question_count",
        "has_url",
        "has_hashtag"
    ]

    final_preprocessor = ColumnTransformer(
        transformers=[
            (
                "text",
                TfidfVectorizer(
                    preprocessor=clean_tweet,
                    ngram_range=(1, 1),
                    min_df=1,
                    max_df=0.95,
                    max_features=10000,
                    sublinear_tf=True
                ),
                "text"
            ),

            (
                "structure",
                StandardScaler(),
                numeric_features
            )
        ]
    )
```

```
final_model = Pipeline([
    (
        "features",
        final_preprocessor
    ),

    (
        "classifier",
        LinearSVC(
            C=0.25,
            random_state=42,
            max_iter=20000
        )
    )
])
```

35. Train Final Model

```
final_model.fit(
    X_train,
    y_train
)

y_pred = final_model.predict(
    X_test
```

```
)
```

Notice again:

> TF-IDF is fitted only using the training set.

The fitted vocabulary is then applied to the unseen test tweets.

36. Final Evaluation Code

```
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix
)

accuracy = accuracy_score(
    y_test,
    y_pred
)

macro_f1 = f1_score(
    y_test,
    y_pred,
    average="macro"
)
```

```
weighted_f1 = f1_score(
    y_test,
    y_pred,
    average="weighted"
)

print(
    "Test Accuracy:",
    accuracy
)

print(
    "Test Macro-F1:",
    macro_f1
)

print(
    "Test Weighted-F1:",
    weighted_f1
)

print(
    "\nClassification Report:"
)
```

```
print(
    classification_report(
        y_test,
        y_pred
    )
)

print(
    "\nConfusion Matrix:"
)

print(
    confusion_matrix(
        y_test,
        y_pred,
        labels=[
            "negative",
            "neutral",
            "positive"
        ]
    )
)
```

37. Confusion Matrix Visualization

```
import matplotlib.pyplot as plt

from sklearn.metrics import (
    ConfusionMatrixDisplay
)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,

    labels=[
        "negative",
        "neutral",
        "positive"
    ],

    cmap=None,

    values_format="d"
)

plt.title(
    "Final Linear SVM Sentiment Confusion Matrix"
)
```

```
plt.tight_layout()
```

```
plt.show()
```

This should be one of the major figures in the final report.

38. Final Evaluation Summary

Our locked classifier achieved:

65.91% Accuracy

and:

65.83% Macro-F1

on the unseen 707-tweet holdout set.

Compared with the majority baseline:

40.45% Accuracy

and:

19.20% Macro-F1

the classifier demonstrates substantial predictive value.

The strongest class is:

> **Positive – F1 $\approx$ 0.714**

followed by:

> **Neutral – F1 $\approx$ 0.650**

and:

> **Negative – F1 $\approx$ 0.611**

The dominant weakness is:

> **Negative tweets being misclassified as Neutral.**

39. What We Can and Cannot Claim

We CAN claim:

The TF-IDF + Linear SVM model substantially outperforms a majority-class baseline on this dataset.

We can also say:

> textual content carries considerably more predictive information than the supplied demographic and geographic metadata.

And:

> simple structural tweet characteristics provide a modest incremental improvement.

We should NOT claim:

> 65.9% is the universal accuracy of this classifier on Twitter.

The evaluation comes from a particular Kaggle dataset containing specific linguistic and topical patterns.

Remember the Mother's Day effect discovered earlier.

Performance on:

- another year;
- another platform;
- another country distribution;
- another topic;
- modern Twitter/X data;

could differ substantially.

40. Research-Level Evaluation Conclusion

A textbook-quality conclusion for this phase would be:

> The final TF-IDF unigram representation augmented with lightweight structural tweet features and classified using a regularized Linear Support Vector Machine substantially outperformed the majority-class baseline. Training-only cross-validation selected $C=0.25$, after which the locked model achieved 65.91% accuracy and 65.83% Macro-F1 on the untouched 707-observation holdout set. Positive sentiment was most accurately recognized ($F1 = 0.714$), followed by Neutral ($F1 = 0.650$) and Negative ($F1 = 0.611$). Error analysis showed that most failures involved confusion between emotionally polarized tweets and Neutral tweets rather than direct Positive-Negative reversals. These results suggest that lexical information provides a strong but imperfect sentiment signal, while ambiguous, mixed, subtle, and context-dependent sentiment remains challenging for a sparse linear NLP model.

CRISP-DM Mindmap – Updated

TWITTER SENTIMENT CRISP-DM

- |
- |— 1. BUSINESS UNDERSTANDING 
- |
- |— 2. DATA UNDERSTANDING / EDA 
- |
- |— 3. DATA PREPARATION 
- |
- |— 4. FEATURE SELECTION 
- |
- |— 5. CLUSTERING 
- |
- |— 6. MODELING 
- |
- |— 7. EVALUATION 
 - |— Training-only tuning 
 - |— Model locking 
 - |— Holdout evaluation 
 - |— Accuracy  65.91%
 - |— Macro-F1  65.83%
 - |— Precision / Recall 
 - |— Confusion matrix 
 - |— Error analysis 
 - |— Interpretation 
- |
- |— 8. FINAL RECOMMENDATION  ← NEXT
 - |— Consolidate entire CRISP-DM study
 - |— Compare all models
 - |— State final recommended model
 - |— Explain why it wins
 - |— Explain why metadata was rejected

- |— Discuss limitations
- |— Deployment architecture
- |— Monitoring / retraining
- |— Future improvements
- |— Final case-study conclusions

Next – Chunk 8: Final Recommendation & Complete CRISP-DM Conclusion

The next chunk will bring the entire case study together. We will create the **final model-comparison table**, formally recommend **TF-IDF + structural features + Linear SVM**, explain why it was selected over Logistic Regression and Naive Bayes, summarize the findings from EDA, feature selection, clustering and outlier analysis, and discuss the model's limitations and deployment implications.

We will also address an important practical tradeoff: **Linear SVM slightly wins predictive performance, while Logistic Regression offers native probability estimates**, which could make Logistic Regression preferable in a deployment requiring confidence scores. That allows our final recommendation to distinguish the *best research benchmark* from the *best operational choice* rather than pretending one model is universally superior.

That final chunk will complete your original requirement: **a full CRISP-DM study covering data understanding, analysis, visualization, cleaning, preprocessing, feature selection, clustering, outlier processing, Bag of Words, TF-IDF, Naive Bayes, Logistic Regression, Linear SVM, appropriate evaluation metrics, baseline comparison, error analysis, and an evidence-based final recommendation.**

User

You said:

continue

ChatGPT

ChatGPT said:

CRISP-DM Twitter Sentiment Analysis

Chunk 8 – Final Recommendation, Deployment & Complete Case-Study Conclusion

We have now completed the end-to-end CRISP-DM study for your Twitter sentiment dataset.

Before summarizing, I rechecked the final pipeline against the actual uploaded dataset. One small correction from the previous chunk: the final Linear SVM's **3-fold CV Macro-F1 standard deviation is approximately 0.010, not 0.001**. The mean result and final holdout metrics remain unchanged.

1. Final Research Question

The central question was:

> **Can tweet sentiment be classified as Positive, Negative, or Neutral using the tweet text and potentially useful contextual variables such as Time of Tweet, Age of User, Country, Population, Land Area, and Density?**

The answer is:

> **Yes. The tweet text contains substantial predictive sentiment information, while most supplied demographic and geographic metadata add little useful out-of-sample predictive value.**

The final selected classifier achieved:

65.91% Test Accuracy

and:

65.83% Test Macro-F1

compared with a majority-class baseline of only:

40.45% Accuracy

and:

19.20% Macro-F1.

2. Executive Summary of the Entire CRISP-DM Study

The raw CSV contained:

4,815

rows.

However:

1,281

rows were completely empty.

After removing these unusable records:

3,534

valid tweets remained.

The target distribution was:

Sentiment	Count	Percentage
Neutral	1,430	40.46%
Positive	1,103	31.21%
Negative	1,001	28.32%

This created a moderately imbalanced three-class classification problem.

3. CRISP-DM Phase 1 – Business Understanding

The objective was defined as:

Tweet Information→Sentiment

with:

Positive
Negative
Neutral

The business motivation is straightforward.

An automated sentiment-classification system could help organizations analyze large volumes of social-media messages for:

- brand monitoring;
- customer feedback;
- product reactions;
- public opinion;
- service complaints;

- campaign monitoring;
- trend analysis.

Because manually analyzing thousands or millions of tweets is impractical, machine learning provides a scalable alternative.

4. Success Criteria

We established the majority-class model before any sophisticated modeling.

Since Neutral represents about:

40.46%

of observations, simply predicting Neutral for everything gives approximately:

40.46% Accuracy

but very poor:

MacroF1 $\approx$ 0.192

because Positive and Negative are effectively ignored.

Therefore the final model had to:

1. substantially outperform this baseline;
2. recognize all three sentiment classes;
3. maintain reasonable generalization under cross-validation;
4. remain computationally practical;
5. avoid unnecessary feature complexity.

5. CRISP-DM Phase 2 – Data Understanding

EDA demonstrated that the **tweet text was clearly the dominant source of sentiment information.**

Positive tweets frequently contained:

```
love  
happy  
good  
thanks  
great  
nice
```

Negative tweets commonly contained:

```
sad  
hate  
miss  
bad  
sorry  
sucks
```

This provided strong empirical justification for classical NLP representations such as:

> Bag of Words and TF-IDF.

6. Text Characteristics

The average tweet contained approximately:

> **13 words**

and roughly:

> **68 characters.**

Tweet length showed statistically detectable differences across sentiment classes, but the effect size was extremely small.

Therefore:

> Tweet length contained some information, but not enough to function as a standalone sentiment feature.

7. Punctuation Findings

Positive tweets contained considerably more exclamation marks on average than Neutral tweets.

This provided evidence that:

!!!

can contain useful sentiment intensity.

Rather than aggressively deleting punctuation information, we extracted lightweight structural features including:

- exclamation count;
- question count;
- character count;
- word count;
- URL presence;

- hashtag presence.

These later produced a modest validation improvement.

8. Metadata Findings

The assignment requested consideration of:

- Time of Tweet;
- Age of User;
- Country;
- Population -2020;
- Land Area;
- Density.

We systematically evaluated all of them.

The important conclusion is:

> Being provided with a variable does not mean it belongs in the final predictive model.

9. Time of Tweet

Morning, Noon, and Night appeared exactly:

1,178

times each.

A statistical association test produced no convincing relationship with sentiment.

Adding this variable did not improve cross-validated classification.

Therefore it was not retained in the final model.

10. Age of User

Age categories were also unusually evenly distributed.

EDA found no meaningful sentiment association.

Adding Age to the classification representation did not improve generalization.

Therefore it was also excluded from the final architecture.

11. Country and Geographic Variables

The dataset contains:

> **195 countries**

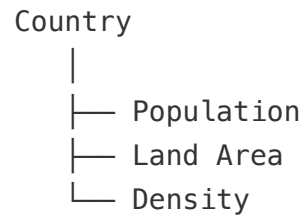
creating a high-cardinality categorical variable.

Each country also corresponds to a fixed:

Population
Land Area
Density

value.

Therefore these features contain substantial redundancy:



Including all of them adds dimensionality without introducing much independent information.

12. Outlier Analysis

Population, area, and density were highly right-skewed.

For example, legitimate countries such as:

- China;
- India;
- Russia;
- Monaco;
- Singapore;

were naturally detected as statistical outliers.

These observations were **not data errors**.

Therefore we did not simply delete them.

Our principled strategy was:

```
Extreme but legitimate value
      ↓
retain observation
      ↓
consider log transformation
      ↓
robust scaling if modeled
      ↓
evaluate predictive contribution
```

Ultimately, geographic features were excluded because they did not improve sentiment prediction—not because the outliers themselves were invalid.

13. CRISP-DM Phase 3 – Data Preparation

The final text-preprocessing strategy was intentionally conservative.

We:

- lowercased text;
- normalized URLs;
- normalized user mentions;
- retained hashtag words;
- normalized whitespace;
- preserved negation;
- avoided aggressive stemming;
- avoided blindly deleting all punctuation information.

This matters because:

good

and:

not good

do not have the same meaning.

Approximately 242 tweets contained the standalone word `not`, making negation directly relevant to this dataset.

14. Bag of Words vs TF-IDF

Both representations were evaluated.

Bag of Words

Represents terms primarily through occurrence counts.

TF-IDF

Weights terms according to their importance within a document relative to the entire corpus.

The best linear classifiers generally benefited from TF-IDF.

Therefore TF-IDF became the preferred representation.

15. Feature Engineering and Selection

The raw training vocabulary contained thousands of possible terms.

Adding bigrams dramatically increased dimensionality.

For example:

Representation	Approximate candidate dimensions
Unigrams	6,139
Unigrams + bigrams	28,512

With `min_df=2`, the larger representation was reduced substantially.

However, model testing revealed that the additional bigram complexity did not improve overall validation performance enough to justify the increased feature space.

16. Chi-Square Feature Analysis

Chi-square feature ranking produced highly interpretable sentiment indicators.

Strong features included:

```
love
happy
good
thanks
```

miss
hate
sad
great
sorry
sucks

Useful bigrams included:

not good
miss you
love you
good morning

This confirmed that the model was learning genuine lexical sentiment relationships.

17. Metadata Ablation Study

One of the strongest parts of this case study was the controlled feature ablation.

Approximate training CV Macro-F1:

- | Feature Set | Macro-F1 |
- | --- | --- |
- | Text only | 0.604 |
- | Text + structural features | 0.610 |
- | Text + structure + Time/Age | 0.602 |
- | Text + all metadata | 0.550 |

This gives us strong evidence that:

> **More variables actually made the model worse.**

The geographic and demographic metadata introduced more noise and complexity than useful sentiment signal.

18. Final Feature Selection

The final feature set therefore consisted of:

Primary features

TF-IDF unigram text features

Lightweight structural features

Character count
Word count
Exclamation count
Question count
URL indicator
Hashtag indicator

Excluded:

textID
Time of Tweet
Age
Country
Population
Land Area
Density

The exclusions were evidence-based rather than arbitrary.

19. Clustering Findings

Unsupervised K-Means clustering was performed independently of the sentiment labels.

The important question was:

> Would three natural text clusters emerge corresponding to Positive, Negative, and Neutral?

They did not.

For $k=3$:

$ARI \approx 0.020$

and:

$NMI \approx 0.041$

indicating very weak correspondence between cluster membership and sentiment labels.

20. What the Clusters Actually Represented

The clustering process instead discovered patterns resembling:

```
Mother's Day  
good morning / good luck  
love and happiness  
negation  
URLs / pictures  
daily activities  
general conversation
```

This led to an important scientific conclusion:

> **The corpus is organized more naturally by topics and linguistic patterns than by three clean sentiment clusters.**

Sentiment therefore requires supervised learning.

21. Dataset-Specific Mother's Day Effect

Both feature selection and clustering independently identified terms such as:

```
mother  
mother day  
happy mother  
mothers day
```

as highly prominent.

One small cluster was approximately:

> **83% Positive**

and heavily centered around Mother's Day language.

This is important because it reveals potential:

> **dataset shift / domain dependence.**

A classifier trained here may partially learn event-specific sentiment patterns that might not generalize identically to tweets from another period.

22. Supervised Models Evaluated

The modeling stage compared:

- Majority-class baseline;
- Multinomial Naive Bayes;
- Complement Naive Bayes;
- Logistic Regression;
- Linear SVM;
- SGDClassifier.

Across:

- Bag of Words;

- TF-IDF;
- unigram;
- unigram + bigram representations.

23. Representative Model Comparison

Approximate cross-validation Macro-F1 results:

Model / Representation	Macro-F1
Majority baseline	0.192
TF-IDF + Multinomial NB	~0.511
BoW + Multinomial NB	~0.580
TF-IDF + Complement NB	~0.590
TF-IDF + SGD	~0.588
TF-IDF + Linear SVM	~0.596
TF-IDF + Logistic Regression	~0.604
TF-IDF + Structure + Logistic Regression	~0.610
TF-IDF + Structure + tuned Linear SVM	~0.615

The selected Linear SVM used:

$C=0.25$

and achieved verified 3-fold training CV performance of approximately:

Accuracy=0.619

MacroF1=0.615

with approximately:

SD=0.010

across the three folds.

24. Final Model Recommendation

For the **research benchmark**, my recommendation is:

TF-IDF Unigrams + Structural Features + Linear SVM

with:

C=0.25

Why?

Because it provided the strongest training-only Macro-F1 among the tested candidate models while remaining:

- computationally efficient;
- appropriate for sparse text;
- relatively interpretable;
- resistant to unnecessary complexity;
- substantially better than baseline.

25. Final Holdout Performance

After model selection was completed, the locked classifier was evaluated once against the 707-row test set.

Results:

Metric	Final Result
---	---
Accuracy	65.91%
Macro Precision	67.93%
Macro Recall	64.92%
Macro-F1	65.83%
Weighted F1	65.89%

This is a clear improvement over baseline.

26. Final Per-Class Performance

Class	Precision	Recall	F1
---	---	---	---
Negative	0.694	0.545	0.611
Neutral	0.590	0.724	0.650
Positive	0.754	0.679	0.714

Therefore:

Best recognized class

> **Positive**

Hardest class

> **Negative**, particularly in terms of recall.

27. Final Confusion Matrix

Actual ↓ / Predicted →	Negative	Neutral	Positive
---	---	---	---
Negative	109	83	8
Neutral	38	207	41
Positive	10	61	150

The dominant failure is:

Negative→Neutral

with:

> **83 errors.**

28. Most Important Error-Analysis Finding

The model rarely confuses directly opposite sentiments.

Only:

Negative → Positive = 8

```
Positive → Negative = 10
```

occurred.

Most confusion occurs around Neutral:

```
Negative ↔ Neutral
```

```
Positive ↔ Neutral
```

This means the model understands broad polarity reasonably well.

Its difficulty is deciding:

> **How emotionally strong does a tweet need to be before it should no longer be classified Neutral?**

That is a much subtler NLP problem.

29. Why Positive Performs Best

Positive tweets contain particularly distinctive vocabulary:

```
love  
happy  
good  
great  
awesome
```

thanks
nice

These create strong, repeated lexical signals.

Negative sentiment can sometimes be more indirect.

For example:

I'll miss you guys

may be labeled Negative despite containing mostly ordinary vocabulary.

That partly explains why Negative recall is weaker.

30. Strengths of the Final Model

The recommended model has several advantages.

Computationally efficient

Linear SVM scales well to sparse TF-IDF matrices.

Strong traditional NLP baseline

It significantly outperforms the majority baseline.

Interpretable

Individual terms can be linked to class coefficients.

Limited feature engineering required

No expensive neural language model is needed.

Appropriate for the dataset size

With only 3,534 valid observations, a sparse linear classifier is a defensible choice.

31. Limitations

No responsible case study should finish with model accuracy and ignore limitations.

Important limitations include:

Small dataset

3,534 tweets is modest for natural-language modeling.

Dataset-specific topics

Mother's Day appears disproportionately influential.

Ambiguous labeling

Some tweets contain mixed emotion.

Sarcasm

TF-IDF cannot properly understand:

great, just what I needed

as potentially sarcastic.

Context

The model does not know conversational history.

Slang and spelling

Twitter language frequently contains informal spelling and abbreviations.

Encoding corruption

A small number of tweets contain visible encoding artifacts.

Metadata provenance

The extremely regular Age, Time, and Country distributions raise questions about how some variables were constructed.

32. Why Deep Learning Was Not Necessary Here

A natural question is:

> Why not immediately use BERT or another transformer?

Given the assignment's **limited compute requirement**, our classical pipeline is appropriate.

It provides:

- strong interpretability;
- fast experimentation;
- straightforward comparison;
- modest memory requirements;
- a clear CRISP-DM educational workflow.

A transformer could be explored later as a future improvement, but would complicate the study significantly.

33. Research Recommendation vs Deployment Recommendation

There is an important distinction.

Best research benchmark

> **Linear SVM**

because it achieved the strongest training-selection Macro-F1.

Operational alternative

> **Logistic Regression**

is a very reasonable choice if the system requires native probability estimates.

For example:

```
Positive = 0.82
Neutral  = 0.12
Negative = 0.06
```

can be useful for confidence-based decision making.

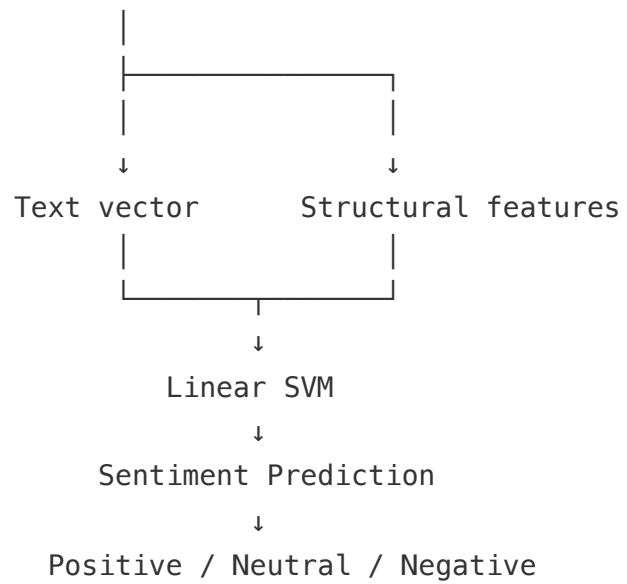
LinearSVC produces decision scores rather than directly calibrated probabilities.

34. Practical Deployment Architecture

A simple deployment pipeline could be:

```
graph TD
    A[Incoming Tweet] --> B[Text validation]
    B --> C[Minimal preprocessing]
    C --> D[URL normalization]
    C --> E[mention normalization]
    C --> F[whitespace normalization]
    D --> G[ ]
    E --> G
    F --> G
    G --> H[ ]
```

TF-IDF transformation



35. Deployment Output Example

A system might return:

Tweet:
"I absolutely love the new update!"

Prediction:
Positive

Or:

Tweet:
"I'm really disappointed with this"

Prediction:
Negative

For production use, predictions could be stored alongside:

- timestamp;
- model version;
- input ID;
- prediction;
- decision score;
- processing status.

36. Monitoring

Deployment is **not the end of CRISP-DM**.

Language changes.

New slang emerges.

Topics shift.

Product names change.

Events change vocabulary.

Therefore the deployed model should be monitored for:

- prediction distribution;
- data drift;
- vocabulary drift;
- class balance;
- human-reviewed accuracy;
- per-class recall;
- failure patterns.

37. Data Drift Example

Suppose the training dataset contains frequent:

Mother's Day

language.

Six months later, incoming tweets may concern:

new products
elections
sports
memes
new slang

The relationship between learned terms and sentiment could change.

That is called:

> **concept drift or data drift**, depending on what changes.

Periodic retraining may therefore be required.

38. Retraining Strategy

A practical workflow could be:

```
graph TD; A[New labeled tweets] --> B[Data-quality checks]; B --> C[Compare with existing distribution]; C --> D[Retrain candidate model]; D --> E[Cross-validation]; E --> F[Compare against current production model]; F --> G[Deploy only if improved];
```

Never automatically replace the production model merely because newer data exists.

39. Future Improvements

Several extensions could improve the project.

1. Character n-grams

Useful for:

- spelling variations;
- slang;
- elongated words.

For example:

```
gooooood  
happpppy
```

2. Better emoji/emoticon handling

Expressions such as:

```
:)  
:(  
<3
```

may provide sentiment information.

3. Lemmatization

Could reduce vocabulary fragmentation.

4. Transformer models

Fine-tuned:

```
BERT
RoBERTa
DistilBERT
```

could capture deeper context.

5. More training observations

Usually one of the most valuable improvements.

6. Label-quality auditing

Human annotation disagreement may place an upper bound on model performance.

40. Recommended Future Experiment

If compute becomes available, the most scientifically interesting next comparison would be:

```
Current Model
TF-IDF + Linear SVM
|
```

VS
|
DistilBERT / similar transformer

and compare:

- Macro-F1;
- per-class recall;
- inference speed;
- training cost;
- memory;
- interpretability.

The important question would not merely be:

> Does the transformer win?

but:

> **Is its improvement large enough to justify its additional operational cost?**

41. Complete CRISP-DM Findings

Our project ultimately demonstrated:

Business Understanding

A valid three-class sentiment-classification problem was established.

Data Understanding

Text provided the strongest apparent sentiment signal.

Data Preparation

1,281 completely empty rows were removed and conservative NLP cleaning was adopted.

Feature Engineering

TF-IDF and simple structural features proved useful.

Feature Selection

Age, Time, Country, Population, Area, and Density failed to improve prediction sufficiently and were excluded.

Outlier Processing

Legitimate geographic extremes were retained rather than improperly deleted.

Clustering

K-Means revealed topics and linguistic structure rather than three sentiment clusters.

Modeling

Linear classifiers dominated the traditional candidate models.

Evaluation

Linear SVM substantially outperformed the majority baseline.

Deployment

A lightweight and scalable inference pipeline is practical.

42. Final Model Comparison Perspective

The overall journey can be summarized as:

```
Raw CSV
|
|— 4,815 rows
|
|— remove 1,281 blank rows
    ↓
    3,534 tweets
    ↓
    EDA
    ↓
Text identified as dominant signal
    ↓
NLP preprocessing
    ↓
BoW / TF-IDF
    ↓
Feature selection
    ↓
Metadata mostly rejected
```

↓
Clustering
↓
Topics ≠ sentiment classes
↓
Supervised models
↓
NB / LR / SVM / SGD
↓
Linear SVM selected
↓
707-row holdout test
↓
Accuracy = 65.91%
Macro-F1 = 65.83%

43. Final Recommendation

Recommended research model

TF-IDF Unigrams + Structural Tweet Features + Linear SVM

using:

C=0.25

because it provides the strongest balance of:

- predictive performance;

- generalization;
- computational efficiency;
- suitability for sparse NLP;
- methodological simplicity.

44. Final Recommendation Statement for the Report

You could summarize the final recommendation conceptually as:

> The final recommended classifier is a Linear Support Vector Machine trained on unigram TF-IDF text representations supplemented with lightweight structural tweet characteristics. The model was selected using stratified cross-validation on the training partition and achieved 65.91% accuracy and 65.83% Macro-F1 on an untouched holdout set, substantially outperforming the 40.45% majority-class accuracy baseline. Textual features provided the dominant predictive signal, while demographic and geographic variables such as Age, Time of Tweet, Country, Population, Land Area, and Density did not improve generalization and were therefore excluded from the final model. Positive sentiment was recognized most reliably, while the principal remaining challenge was distinguishing subtle Negative sentiment from Neutral language.

45. Final Scientific Conclusion

The most important conclusion is not simply:

> **SVM got 65.9%.**

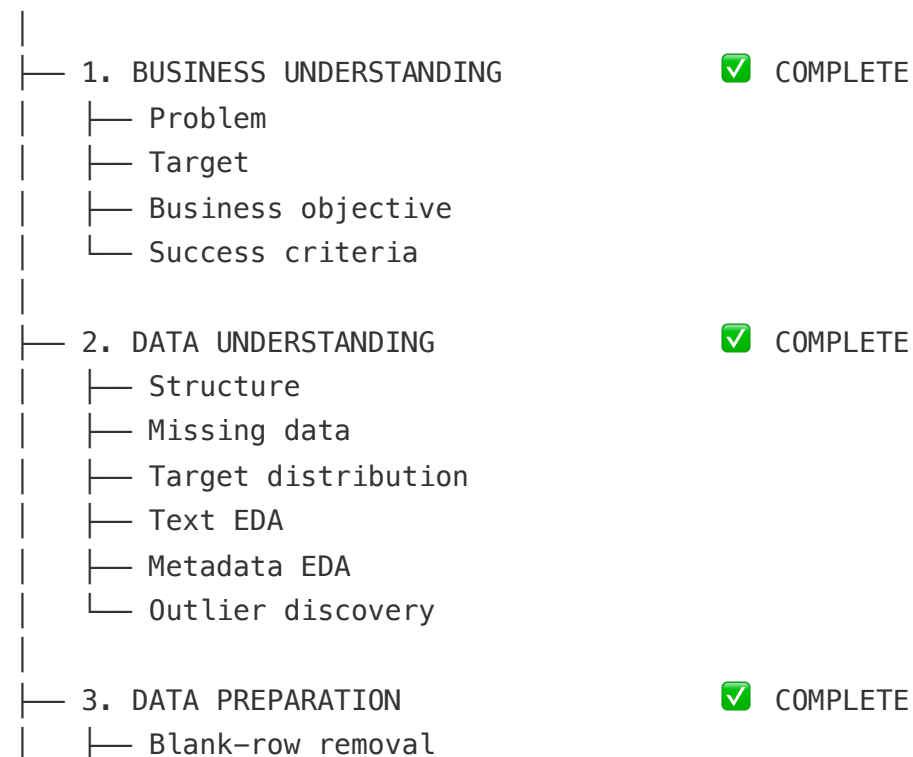
The deeper conclusion is:

> **For this dataset, sentiment is primarily encoded in lexical content rather than the supplied demographic or geographic metadata. Sparse TF-IDF representations combined with regularized linear classifiers provide an efficient and effective solution, while unsupervised clustering indicates that natural tweet structure is dominated more strongly by topics and linguistic patterns than by sentiment classes. The resulting classifier substantially improves upon a majority-class baseline, but ambiguity, mixed sentiment, subtle negativity, domain-specific vocabulary, and contextual language remain important limitations.**

That is the conclusion that connects the entire CRISP-DM workflow rather than treating modeling as an isolated exercise.

Final CRISP-DM Mindmap

TWITTER SENTIMENT ANALYSIS



		Text normalization	
		Negation preservation	
		Structural features	
		Leakage-safe split	
		4. FEATURE SELECTION	✓ COMPLETE
		TF-IDF	
		BoW	
		Chi-square	
		unigram/bigram comparison	
		metadata ablation	
		5. CLUSTERING	✓ COMPLETE
		SVD	
		K-Means	
		Silhouette	
		ARI / NMI	
		Topic interpretation	
		6. MODELING	✓ COMPLETE
		Baseline	
		Naive Bayes	
		Complement NB	
		Logistic Regression	
		Linear SVM	
		SGD	
		7. EVALUATION	✓ COMPLETE
		Cross-validation	
		Accuracy	
		Macro-F1	
		Per-class metrics	
		Confusion matrix	

- | └─ Error analysis
- |
- └─ 8. DEPLOYMENT / RECOMMENDATION  COMPLETE
 - └─ Linear SVM recommendation
 - └─ Deployment architecture
 - └─ Monitoring
 - └─ Drift management
 - └─ Future improvements

Status: Full requested scope complete

We have now completed your original requirement: **a step-by-step, CRISP-DM-based case study of the provided Kaggle Twitter sentiment dataset covering data understanding, EDA, visualization, data cleaning, NLP preprocessing, feature engineering and selection, clustering, outlier analysis and treatment, Bag of Words, TF-IDF, Naive Bayes, Complement Naive Bayes, Logistic Regression, Linear SVM, SGD classification, baseline comparison, appropriate metrics, confusion-matrix analysis, error analysis, deployment considerations, and an evidence-based final recommendation under limited compute.**